

Hospital Management System

A modular, role-based web application for patient, appointment, clinical-record and billing management

Author	Registration No.	Course	Repository
[Your Name]	[Reg. Number]	[Course Code - Name]	[GitHub URL]

1. Overview

The Hospital Management System (HMS) replaces paper registers and scattered spreadsheets in a small or mid-sized clinic with one secure system. Staff can register patients, book doctor appointments without double booking, record diagnoses and prescriptions, generate bills, and view summary reports. Access is controlled by role (Admin, Receptionist, Doctor). The project follows a layered architecture (routes, services, repositories, database) and is covered by automated tests.

2. Features

- **User & access management** - login, three roles, account creation/deactivation, password change (hashed passwords).
- **Patient management** - register, search by name/phone/ID, update, soft-delete, view visit history.
- **Doctor & appointment scheduling** - manage doctors, book/cancel/reschedule, automatic slot-conflict check.
- **Clinical records** - diagnosis, prescription and notes linked to each appointment.
- **Billing** - fee + charges - discount, payment status, printable invoice, one bill per appointment.
- **Reports** - appointment summary, revenue by date range/doctor, department-wise patient load.
- **Quality** - input validation, centralised error handling, rotating log file, pytest test suite.

3. Technologies Used

Area	Technology
Language	Python 3.10+
Web framework	Flask 3.x (Jinja2 templates)
Database	SQLite 3
Security	Werkzeug password hashing, role-based access decorator
Front-end	HTML5, CSS3
Testing	pytest
Version control	Git / GitHub

4. Project Structure

```
hospital-management-system/
|-- app/
|   |-- __init__.py      # application factory
|   |-- config.py        # settings
|   |-- db.py            # DB connection and schema setup
|   |-- routes/          # controllers (auth, patient, appointment, billing, report)
|   |-- services/        # business logic
|   |-- repositories/    # SQL access, one file per table
|   |-- utils/           # validators, security, logger, exceptions
|   |-- templates/       # HTML pages
|   |-- static/          # CSS
|-- data/schema.sql       # tables and indexes
|-- data/seed.sql        # demo data
|-- tests/               # pytest tests
|-- run.py               # entry point
|-- requirements.txt
|-- README.md
`-- statement.md
```

5. Installation & Running

Prerequisites: Python 3.10 or newer, pip and Git.

Step 1 - Clone the repository

```
git clone https://github.com/<your-username>/hospital-management-system.git
cd hospital-management-system
```

Step 2 - Create and activate a virtual environment

```
python -m venv venv

# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate
```

Step 3 - Install dependencies

```
pip install -r requirements.txt
```

Step 4 - Run the application

```
python run.py
```

On first run the database is created from `data/schema.sql` and loaded with demo data from `data/seed.sql`. Open `http://127.0.0.1:5000` in your browser.

Demo accounts (from the seed data - change these before any real use)

Role	Username	Password
Admin	[admin]	[set in seed.sql]
Receptionist	[reception]	[set in seed.sql]
Doctor	[doctor1]	[set in seed.sql]

6. Usage Guide

- Log in as **Receptionist** -> *Patients* -> *Register* to add a patient.

- Go to *Appointments* -> *Book*, choose patient, doctor, date and time. A taken slot is rejected.
- Log in as **Doctor** -> open the appointment -> enter diagnosis and prescription.
- As **Receptionist** -> *Billing* -> generate the bill and record payment.
- As **Admin** -> *Reports* to view appointments and revenue; *Users* to manage accounts.

7. Running the Tests

```
pytest -v                # run all tests
pytest tests/test_billing.py  # run one test file
```

Tests use a temporary in-memory database, so your real data is never touched. They cover login and role restrictions, patient validation, double-booking prevention, bill calculation, and SQL-injection safety.

8. Configuration & Logs

Settings (secret key, database path, login-attempt limit) are in `app/config.py`. Set a strong `SECRET_KEY` before deployment. Logs are written to `logs/hms.log` (rotating).

9. Future Enhancements

Patient portal, SMS/email reminders, inpatient bed management, pharmacy and lab modules, insurance claims, PostgreSQL/Docker deployment, and two-factor authentication.

10. Author

[Your Name] - [Reg. Number] - VITyarthi Build Your Own Project, 2026